

Worksheet No. - 05

Student Name: Sukhleen Kaur

UID: 25MCA20031

Branch: MCA

Section/Group: 25MCA-6B

Semester: 2nd

Date of Performance: 26.03.2026

Subject Name: Design and Analysis of Algorithm Lab **Subject Code:** 25CAP-612

Aim/Overview of the practical: (A). Implement 0/1 Knapsack problem using dynamic programming.

Objective: To implement the 0/1 Knapsack problem using dynamic programming in C in order to determine the maximum value that can be obtained within a given weight capacity by selecting optimal items without repetition.

Procedure/Algorithm/Code

Algorithm

1. Start
2. Input number of items n , weights array $wt[]$, values array $val[]$, and capacity W .
3. Create a DP table $dp[n+1][W+1]$.
4. Initialize first row and column with 0 (base case).
5. For each item $i = 1$ to n :
 - For each capacity $w = 1$ to W :
 - If $wt[i-1] \leq w$:
 - $dp[i][w] = \max(val[i-1] + dp[i-1][w - wt[i-1]], dp[i-1][w])$
 - Else:
 - $dp[i][w] = dp[i-1][w]$
6. The final answer is $dp[n][W]$.
7. Stop.

Code

```
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int knapsack(int W, int wt[], int val[], int n) {
    int i, w;

    int dp[n+1][W+1];

    for (i = 0; i <= n; i++) {
        for (w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0; // Base case
            else if (wt[i-1] <= w)
                dp[i][w] = max(val[i-1] + dp[i-1][w - wt[i-1]], dp[i-1][w]);
            else
                dp[i][w] = dp[i-1][w];
        }
    }

    return dp[n][W];
}

int main() {
    int n, W;

    printf("Enter number of items: ");

    scanf("%d", &n);

    int val[n], wt[n];
```

```
printf("Enter values of items:\n");

for (int i = 0; i < n; i++) {
    scanf("%d", &val[i]);
}

printf("Enter weights of items:\n");

for (int i = 0; i < n; i++) {
    scanf("%d", &wt[i]);
}

printf("Enter maximum capacity of knapsack: ");
scanf("%d", &W);

printf("Maximum value in knapsack = %d\n", knapsack(W, wt, val, n));

return 0;
}
```

Output

```
Enter number of items: 4
Enter values of items:
50 70 110 130
Enter weights of items:
20 30 40 50
Enter maximum capacity of knapsack: 80
Maximum value in knapsack = 200
```

Learning outcomes (What I have learnt):

1. Understand the concept of 0/1 Knapsack problem.
2. Learn how dynamic programming is used to solve optimization problems.
3. Develop skills in 2D array (DP table) implementation

Aim/Overview of the practical: (B). Compute the transitive closure of a given directed graph using Warshall's algorithm.

Objective: The objective of this program is to compute the transitive closure of a directed graph using Warshall's Algorithm. It aims to determine the reachability between all pairs of vertices by identifying both direct and indirect paths in the graph. The program uses an adjacency matrix representation and systematically updates it to reflect whether a path exists between any two vertices. This helps in understanding graph connectivity and the concept of path existence in directed graphs.

Procedure/Algorithm/Code

Algorithm

1. Start
2. Input the number of vertices n.
3. Input the adjacency matrix $A[n][n]$ of the directed graph.
4. Initialize the transitive closure matrix $T = A$.
5. Use a loop for each intermediate vertex k from 0 to n-1:
 - For each vertex i from 0 to n-1:
 - For each vertex j from 0 to n-1:
 - Update:
$$T[i][j] = T[i][j] \text{ OR } (T[i][k] \text{ AND } T[k][j])$$
6. After completing all iterations, matrix T will contain the transitive closure.
7. Display the matrix T.
8. Stop.

Code:

```
#include <stdio.h>

#define MAX 10

int main() {

    int n, i, j, k;

    int graph[MAX][MAX];
```

```
printf("Enter number of vertices: ");

scanf("%d", &n);

printf("Enter the adjacency matrix:\n");

for(i = 0; i < n; i++) {
    for(j = 0; j < n; j++) {
        scanf("%d", &graph[i][j]);
    }
}

for(k = 0; k < n; k++) {
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            graph[i][j] = graph[i][j] || (graph[i][k] && graph[k][j]);
        }
    }
}

printf("\nTransitive Closure Matrix:\n");

for(i = 0; i < n; i++) {
    for(j = 0; j < n; j++) {
        printf("%d ", graph[i][j]);
    }

    printf("\n");
}

return 0;
}
```

Output

```
Enter number of vertices: 4
```

```
Enter the adjacency matrix:
```

```
0 0 0 0
```

```
0 0 1 0
```

```
0 0 0 1
```

```
1 0 0 0
```

```
Transitive Closure Matrix:
```

```
0 0 0 0
```

```
1 0 1 1
```

```
1 0 0 1
```

```
1 0 0 0
```

Learning outcomes (What I have learnt):

1. Understood the concept of transitive closure in directed graphs.
2. Learnt how to apply Warshall's Algorithm to find reachability.
3. Represented graphs using an adjacency matrix.
4. Identified direct and indirect paths between vertices.